
BaseDyGraph: An Interlaced Dynamic Graph Benchmark and Model for Controlled Spatio-Temporal Ablations

Autonomous-Fox Laboratories
Internal Research and Training Note
research@autonomous-fox.ai

Abstract

This note describes BaseDyGraph, a controlled synthetic benchmark and interlaced spatio-temporal graph model for training new researchers to run principled ablations on production-style dynamic graph architectures. The benchmark consists of discrete node states evolving under a latent regime process, where each regime carries a sparse directed graph. The model must predict the next state at every node and, where possible, recover the active graph. Because the data generator exposes both static and dynamic oracle baselines, experiments can be interpreted in terms of dynamic headroom: the performance available only to models that infer time-varying structure. The architecture combines causal per-node temporal encoding, static and dynamic graph scorers, GAT-style value aggregation (message passing in which each node forms a weighted average of its neighbours' values), interlaced spatio-temporal blocks (alternating within-node time processing and across-node graph processing), a convex dynamic-graph gate (a learned mixture weight between a stable average graph and the time-varying one), and graph-shape regularisation. Readers new to the system should begin with Section 2, which gives a plain-language mental model, the prerequisites, an architecture diagram, and a map from the notation in this note to the objects in the codebase. The document is written as a teaching tool and technical reference: it gives the mathematical model, the implementation-level modules, the diagnostic quantities, and the ablation logic needed to understand why particular architectural and optimisation choices matter.

1 Purpose and scope

BaseDyGraph is an internal experimental system for studying dynamic graph learning under conditions where the ground-truth graph is known. The objective is not simply to produce a high accuracy number. The objective is to train researchers to ask controlled questions about architecture, optimisation, and diagnostics. In particular, the benchmark is designed to answer questions of the following form:

- Can a learned dynamic graph outperform a static graph when the true dependency structure changes over time?
- Does the inferred adjacency recover the true regime graph, or does it only improve prediction as an uninterpretable attention map?
- Which parts of the architecture help extract the dynamic-only headroom?
- Which training tricks stabilise dynamic graph learning without destroying graph recovery?
- How should graph entropy, graph correlation, oracle accuracy, and predictive loss be read together?

The note is written as a teaching tool. It is meant to be read alongside the accompanying notebook by someone learning to run principled experiments on dynamic graph models, and the discipline it builds transfers directly to larger production systems. The conventions below therefore mix formal notation with interpretation paragraphs: the notation makes the model unambiguous, and the interpretation paragraphs explain why the design choices were made and how to read the results.

Learning objectives. By working through the benchmark a reader should learn to:

- run a model end to end — data generation, training, and evaluation — and reproduce a known result before changing anything;
- design and run controlled ablation studies, changing one architectural or optimisation factor at a time;
- run hyperparameter sweeps and read them without being misled by run-to-run noise;
- build intuition for how architecture choices (graph activation, what the spatial layer mixes, interlacing, the dynamic-base gate, regularisation) affect both prediction and graph recovery;
- read several metrics jointly — accuracy against the oracle, graph correlation, AUROC, entropy, and the α trajectory — rather than optimising a single number;
- use oracle and headroom diagnostics to bound what is achievable and interpret results relative to that bound, a habit that transfers to real problems where no oracle exists;
- recognise characteristic failure modes, such as a dynamic graph collapsing to static behaviour, dead gradients from an over-sparse activation, or a graph that ranks edges well yet stays too diffuse to sharpen predictions.

Central principle. The benchmark provides a known dynamic graph. We can therefore compute how well an oracle static graph would do, how well an oracle dynamic graph would do, and how much of the remaining dynamic headroom a learned model captures. This is the main reason the system is useful as a training tool.

2 Mental model and prerequisites

This section is the recommended entry point. It states, in plain language, what the system does and what background a reader needs, before the formal definitions begin in Section 3.

The system in one paragraph. BaseDyGraph is a controlled laboratory for one question: *when the rule that couples a set of interacting units changes over time, can a model discover that changing structure from data, and does discovering it actually help prediction?* The data are N nodes, each holding a discrete state that updates every timestep. A node’s next state is influenced by the current states of a subset of other nodes — its neighbours in a directed graph. The catch is that this graph is not fixed: a hidden *regime* selects which graph is active, and the regime switches occasionally. A model that assumes one fixed graph (the *static* view) can only capture the average coupling; a model that infers the currently active graph at each step (the *dynamic* view) can in principle do much better. The gap between those two is the *dynamic headroom*, and the point of the benchmark is that we can measure it exactly, because we built the generator and therefore know the true graphs.

The task. The reader — and the model — is given only the observed state sequences $s_{1:T}$ for all N nodes. The regime path $r_{1:T}$ and the per-regime graphs $A^{(r)}$ are *hidden*: they are used only to build the oracles and to score graph recovery, never as model inputs. From the states alone, the model must

1. predict each node’s next state s_{t+1} at every timestep — the primary objective, scored by accuracy and NLL and read against the static and dynamic oracles; and
2. optionally expose, at each timestep, a per-node adjacency that recovers the currently active regime graph — scored by correlation and AUROC against the truth, and characterised by its entropy.

The reader’s job is to find model configurations and training procedures that capture as much of the dynamic headroom as possible while keeping the recovered graph both faithful (high correlation and AUROC) and selective (low entropy). Predicting well without recovering structure, or recovering a diffuse graph that does not sharpen predictions, are informative partial outcomes rather than the goal.

What a strong solution looks like. The problem is deliberately calibrated to be moderately hard but achievable. We believe that with the right configuration a learned model can come close to the dynamic-oracle accuracy ceiling while recovering a faithful, selective graph. As a rough target to aim for: graph correlation above 0.8, edge AUROC around 0.9, and mean graph entropy below 2 (against a true-graph entropy of about 1.78, Section 3). These are goals, not guarantees; reaching them is the purpose of the ablation and sweep programme in Sections 15–16.

Why both a temporal and a spatial stage. Two different kinds of information matter. *Temporal* processing looks along each node’s own history to infer which regime is likely active now; *spatial* processing routes information between nodes along the inferred graph. Neither alone is enough: time-only processing never exchanges information across nodes, and graph-only processing has no way to tell which regime it is in. The architecture therefore alternates the two, as sketched in Figure 1.

Prerequisites. A reader should be comfortable with multiclass classification and cross-entropy; the Transformer encoder and causal (autoregressive) masking; and the basic idea of attention as a learned, normalised weighting over a set. Familiarity with graph attention networks (GAT) is helpful but not assumed — the one fact needed is that a GAT layer updates each node by taking a weighted average of its neighbours, where the weights form an adjacency matrix. Here that adjacency is not given: it is *inferred* by a graph scorer, and is then a first-class object that we log, regularise, and compare against the truth.

Key terms, in brief. Full definitions are in the glossary (Appendix A); the four that recur most are:

- **Regime:** a hidden mode the system is in (think weather or market regime). While a regime lasts, the coupling graph is fixed; it switches occasionally, controlled by stickiness ρ .
- **Static vs. dynamic graph:** a static graph is one fixed adjacency used at all times; a dynamic graph changes with the regime or context.
- **Oracle:** a diagnostic that is *handed* the true graph — either time-averaged (static oracle) or the correct per-step graph (dynamic oracle) — instead of learning it. Oracles give an upper reference for each view.
- **Dynamic headroom:** dynamic-oracle accuracy minus static-oracle accuracy — the performance reachable only by a model that recovers the time-varying structure.

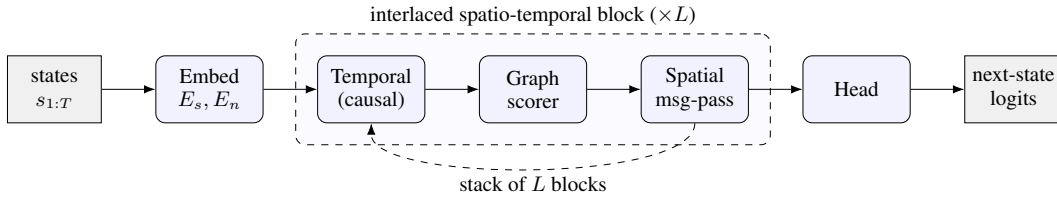


Figure 1: Macro data flow. Discrete states are embedded, then passed through one or more interlaced blocks of temporal (within-node, over time) and spatial (across-node, over the inferred graph) processing, before a per-node next-state head. The graph scorer can be static, dynamic, or dynamic-base with a convex α gate (Sections 7–8); its output adjacency is what the spatial stage aggregates over.

From notation to code. This note is deliberately abstract about the exact generator and orchestration so the equations stay readable; the runnable definitions live in the codebase. Table 1 maps the main symbols and components to the files and objects a reader will open. The newer orchestration features — interlacing, the convex α gate, and graph regularisation — are configured in the current notebook, which is the source of truth for runnable values.

3 Synthetic dynamic graph process

This section defines the data generator: how node states evolve, how the hidden regime selects a graph, and what the next-state distribution looks like. The notation introduced here is reused throughout the note.

Table 1: Map from the notation in this note to objects in the codebase.

Concept	Symbol	Code object (file)
Per-regime true graph	$A^{(r)}$	<code>regime_graphs</code> (<code>synthetic_generators.py</code>)
State-compatibility kernel	ϕ	<code>spatial_kernel</code> (<code>synthetic_generators.py</code>)
Transition / coupling law	η	<code>_coupling_logits, _sample_sequence</code>
State / node embedding	E_s, E_n	<code>state_embedding, node_embedding</code> (<code>model.py</code>)
Causal temporal encoder	Temporal	<code>PerNodeTemporalEncoder</code> (<code>modules.py</code>)
Static graph scorer	A^{static}	<code>StaticGraphScorer</code> (<code>modules.py</code>)
Dynamic graph scorer	A^{dyn}	<code>DynamicGraphScorer</code> (<code>modules.py</code>)
Dynamic-base scorer	$L^{\text{base}} + L^{\text{dyn}}$	<code>DynamicBaseGraphScorer</code> (<code>modules.py</code>)
Edge activations	<code>softmax, etc.</code>	<code>_normalise, _sparsemax, _entmax15</code>
State-pair bias	$B_{h,u,v}$	<code>state_pair_bias</code> (<code>modules.py</code>)
Spatial message passing	$m_{t,i}$	<code>SpatialMessagePassing,</code> <code>_SpatialMPBlock</code>
Next-state head and loss	$\mathcal{L}_{\text{CE}}$	<code>NextStateHead, _shared_step</code> (<code>model.py</code>)
Graph recovery metrics	<code>corr, AUROC</code>	<code>_compute_graph_recovery_metrics</code> (<code>model.py</code>)
Configuration object	—	<code>ModelConfig</code> (<code>utilities.py</code>)
Interlacing, α gate, regularisation	$\alpha, \lambda_H, \lambda_S, \lambda_E$	current notebook (source of truth)

3.1 Objects and notation

Let B denote the number of sampled sequences, N the number of nodes, T the sequence length, K the number of discrete states, and R the number of latent regimes. In the current benchmark configuration,

$$N = 32, \quad K = 32, \quad T = 128, \quad R = 12. \quad (1)$$

For sample b , time t , and node i , the observed node state is

$$s_{b,t,i} \in \{1, \dots, K\}. \quad (2)$$

A latent regime variable

$$r_{b,t} \in \{1, \dots, R\} \quad (3)$$

selects the active regime graph at time t . Each regime r has a row-stochastic directed graph

$$A^{(r)} \in [0, 1]^{N \times N}, \quad \sum_{j=1}^N A_{ij}^{(r)} = 1. \quad (4)$$

The row $A_{i,:}^{(r)}$ specifies which source nodes j influence receiving node i under regime r .

3.2 Regime persistence

The regime process is persistent. With regime stickiness ρ , the next regime usually remains equal to the previous regime and occasionally changes. In the current benchmark,

$$\rho = 0.85, \quad (5)$$

which gives an empirical mean regime dwell time of approximately 6.75 timesteps in the diagnostic batch. This value is important. If regimes switch too quickly, the dynamic oracle can have high headroom while the learned model cannot infer the active graph quickly enough. If regimes persist long enough, the active graph becomes filterable from the sequence.

Interpretation. Regime stickiness controls how long the model has to infer and exploit the active graph. Earlier experiments with very short dwell times produced a large dynamic oracle gap but made the dynamic graph difficult to learn. The current dwell time is deliberately challenging but not absurd. It produces a genuine filtering problem.

3.3 Transition law

The generator constructs next-state probabilities from current node states, a regime-specific graph, and a spatial coupling term. The exact implementation contains several configurable terms, including self-transition strength, regime scale, spatial scale, graph sparsity, and shock probability. Abstractly, the conditional distribution has the form

$$p(s_{b,t+1,i} = k \mid s_{b,t,1:N}, r_{b,t}) = \text{softmax}_k(\eta_{b,t,i,k}), \quad (6)$$

where the logits $\eta_{b,t,i,k}$ include a graph-weighted contribution from the current states of other nodes:

$$\eta_{b,t,i,k} = \eta_{s_{b,t,i},k}^{\text{base}} + \lambda_{\text{spatial}} \sum_{j=1}^N A_{ij}^{(r_{b,t})} \phi_k(s_{b,t,j}) + \eta_{b,t,i,k}^{\text{shock}}. \quad (7)$$

Here λ_{spatial} is the spatial scale and ϕ_k denotes the state-dependent contribution of a source node to target-state logit k . Concretely, ϕ is a fixed random state-compatibility kernel of shape $K \times K$ (the `spatial_kernel` in `synthetic_generators.py`): a neighbour j in state $s_{b,t,j}$ contributes the row $\phi[s_{b,t,j}, :]$, and these rows are aggregated over neighbours with weights $A_{ij}^{(r_{b,t})}$. The exact generator is used for experiments; the equation above states the role of the active graph.

Interpretation. The prediction problem is a K -way next-state classification problem at every node and time. It is not binary classification. Therefore a cross-entropy near 2.5 is not large in this setting. The random uniform entropy over $K = 32$ states is $\log(32) \simeq 3.466$ nats.

3.4 Current data-generation configuration

The current non-sweep notebook uses the configuration in Table 2. These values define the principal benchmark discussed in this note.

Table 2: Current synthetic data configuration.

Quantity	Value
Number of samples	3000
Train / validation / test split	2500/500/0
Sequence length	128
Number of nodes	32
Number of states	32
Number of regimes	12
Regime stickiness	0.85
Regime scale	1.00
Graph sparsity	0.75
Self-transition scale	0.50
Spatial scale	10.00
Shock probability	0.01
Data seed	7

4 Oracle diagnostics and dynamic headroom

The benchmark includes diagnostics that quantify task difficulty before any learned model is trained. These diagnostics are essential because the achievable accuracy is far below 1.0. In the current benchmark, the diagnostic values are shown in Table 3.

4.1 Reading the diagnostics table

Two numbers in Table 3 look almost equal and a new reader should know why. The *achievable accuracy ceiling* (0.3958) is the accuracy of a predictor that knows the true per-step conditional distribution and always picks its mode; it is the irreducible best, capped well below 1.0 because the transition law is genuinely stochastic (the shock term and the softmax sampling inject noise). The

Table 3: Representative data diagnostics for the current benchmark. Values are from the current diagnostic cell. Entropies and NLL values are in nats.

Diagnostic	Value
Achievable accuracy ceiling	0.3958
Persistence rate	0.1031
Oracle NLL	2.0564
True conditional entropy	2.0577
$H(s_{t+1} \mid s_t)$	3.2568
Structure information	1.1991
State entropy	3.2976
Uniform entropy log 32	3.4657
State balance ratio	14.4643
True graph entropy, mean	1.7823
True graph effective neighbours	5.9433
True graph entropy, min / max	0.7790/2.4190
Mean regime dwell	6.7517
Static oracle accuracy	0.2299
Dynamic oracle accuracy	0.3931
Dynamic headroom, accuracy	0.1632
Dynamic headroom, NLL	0.6525

dynamic oracle accuracy (0.3931) is what you get by injecting the true active graph at each step and otherwise training normally. They nearly coincide because, in this benchmark, knowing the active graph is almost all of what knowing the true distribution buys you — which is precisely why recovering the dynamic graph is the central challenge. The static oracle (0.2299) falls far short, and that shortfall is the dynamic headroom.

4.2 Static and dynamic oracles

The static oracle measures the best attainable performance when a model is given the best time-averaged graph rather than the active regime graph. The dynamic oracle measures performance when the true active graph is injected at each timestep. The dynamic headroom is the difference between these two quantities:

$$\Delta_{\text{dyn}}^{\text{acc}} = \text{Acc}_{\text{dyn oracle}} - \text{Acc}_{\text{static oracle}}. \quad (8)$$

For the current benchmark,

$$\Delta_{\text{dyn}}^{\text{acc}} \simeq 0.3931 - 0.2299 = 0.1632. \quad (9)$$

The aim of the ablation programme is to capture as much of this dynamic headroom as possible using learned architectures and training procedures.

Interpretation. The static oracle is already useful because the average graph contains structure. However, a large additional gap remains for models that infer the active time-varying graph. A learned model that reaches, for example, an accuracy of 0.30 is not merely at 30% of a putative perfect classifier. Relative to the static oracle and the dynamic oracle, it has captured a meaningful fraction of the dynamic-only headroom — and a strong configuration should push well beyond that, toward the dynamic ceiling.

4.3 Graph entropy and effective neighbours

For a row-stochastic graph row $A_{i,:}$, graph entropy is

$$H(A_{i,:}) = - \sum_{j=1}^N A_{ij} \log A_{ij}. \quad (10)$$

The effective number of neighbours is

$$n_{\text{eff}} = \exp\{H(A_{i,:})\}. \quad (11)$$

For $N = 32$, the maximum row entropy is $\log 32 \simeq 3.466$. The true graph entropy in the current benchmark is approximately 1.78, corresponding to about six effective neighbours. This fact is crucial for interpreting learned graphs. A learned entropy near 3.3 is almost uniform; an entropy near 2.6 corresponds to about 13.5 effective neighbours; an entropy near 2.14 corresponds to about 8.5 effective neighbours.

Interpretation. Graph correlation alone is not sufficient. A graph may rank true edges well but remain too diffuse to affect top-one prediction. Entropy tells us whether the model is concentrating mass on a selective set of neighbours. As an illustration, in one example run without regularisation entropy fell to about 2.6 with graph correlation around 0.75; in another with target-entropy regularisation it fell further to about 2.14 at similar accuracy and correlation, indicating a cleaner graph at little predictive cost. These are illustrative earlier runs rather than the best configuration found; the target to aim for is sharper still (correlation above 0.8, entropy below 2).

5 Model overview

BaseDyGraph maps a sequence of discrete node states to next-state logits. The model contains the following components:

1. state and optional node embeddings;
2. causal per-node temporal encoding;
3. graph scoring, either static, dynamic, or dynamic-base;
4. GAT-style spatial message passing using the inferred graph;
5. optional interlacing of temporal and spatial blocks;
6. a next-state prediction head;
7. graph diagnostics and optional graph-shape regularisation.

Let $e_{b,t,i} \in \mathbb{R}^d$ denote the input embedding for node i at time t . It is constructed from a state embedding and, when enabled, a node embedding:

$$e_{b,t,i} = E_s[s_{b,t,i}] + E_n[i]. \quad (12)$$

The current high-capacity configuration uses $d = 64$, 4 temporal attention heads, 2 edge heads, and graph hidden dimension 64.

Interpretation. The node embedding lets the model distinguish node identities. The state embedding represents the discrete state. Together they give the temporal encoder information about both what a node is doing and which node is doing it.

6 Causal temporal encoder

The temporal module is a shared causal Transformer encoder applied independently to each node sequence. Given input embeddings $E \in \mathbb{R}^{B \times N \times T \times d}$, each node trajectory is reshaped into a batch of BN sequences and processed with a causal mask:

$$h_{b,1:T,i}^{(\ell)} = \text{TemporalBlock}^{(\ell)}(e_{b,1:T,i}). \quad (13)$$

Causality ensures that $h_{b,t,i}$ depends only on states up to time t , not on future states.

Interpretation. A temporal encoder by itself can model each node’s history, but it cannot directly exchange information across nodes. In the hard setting, the active graph is partly identifiable only from cross-node interaction patterns. This is why temporal-only processing is not enough.

7 Graph scorers

A *graph scorer* is the part of the model that produces the adjacency the spatial stage will use. It answers “who influences whom right now?” and hands the resulting weighted graph downstream. Three variants are defined below, in increasing order of how much they let the graph vary with context.

7.1 Static graph scorer

The static graph scorer learns free edge logits for every edge and head:

$$L_{h,i,j}^{\text{static}} \in \mathbb{R}. \quad (14)$$

The adjacency is obtained by row normalisation:

$$A_{h,i,j}^{\text{static}} = \text{softmax}_j (L_{h,i,j}^{\text{static}}). \quad (15)$$

The same graph is used at all times.

Interpretation. The static graph is a strong baseline. It has a direct identity-indexed parameterisation and can learn the average dependency structure. It is easier to optimise than a fully dynamic graph. A dynamic model should be judged against this baseline, not only against a no-graph model.

7.2 Dynamic attention graph scorer

The dynamic graph scorer constructs time-varying edge logits from contextual hidden states. For each edge head h , node i supplies a query and node j supplies a key:

$$q_{b,t,h,i} = W_h^Q h_{b,t,i}, \quad (16)$$

$$k_{b,t,h,j} = W_h^K h_{b,t,j}. \quad (17)$$

The dynamic logit for edge $j \rightarrow i$ is

$$L_{b,t,h,i,j}^{\text{dyn}} = \frac{q_{b,t,h,i}^\top k_{b,t,h,j}}{\sqrt{d_e}}, \quad (18)$$

where d_e is the per-head edge dimension. The normalised graph is

$$A_{b,t,h,i,j}^{\text{dyn}} = \text{softmax}_j (L_{b,t,h,i,j}^{\text{dyn}}), \quad (19)$$

with optional alternatives such as sparsemax, entmax, or gated normalisation.

Interpretation. This is an attention-parameterised dynamic adjacency. The attention score is not used merely as an explanation after the fact. It is the actual graph through which spatial messages are passed and is evaluated against the true regime graph.

7.3 Dynamic-base graph scorer

The dynamic-base scorer combines a learned static base graph with a dynamic correction:

$$L_{b,t,h,i,j}^{\text{base+dyn}} = L_{h,i,j}^{\text{base}} + L_{b,t,h,i,j}^{\text{dyn}}. \quad (20)$$

This parameterisation is motivated by training stability. Early in training a purely dynamic graph can be noisy. The base graph provides a stable average dependency structure, while the dynamic term learns deviations from that base.

Interpretation. The base graph is not a concession that the task is static. It is an optimisation scaffold. The model can begin with a stable average graph and progressively increase reliance on the dynamic component as the dynamic scorer becomes useful.

7.4 State-pair bias

When enabled, the scorer contains a small pairwise lookup table

$$B_{h,u,v} \in \mathbb{R}, \quad (21)$$

indexed by edge head h , receiver state u , and source state v . For edge $j \rightarrow i$ at time t , the bias is

$$B_{h,s_{b,t,i},s_{b,t,j}}. \quad (22)$$

The corresponding logit becomes

$$L_{b,t,h,i,j} \leftarrow L_{b,t,h,i,j} + B_{h,s_{b,t,i},s_{b,t,j}}. \quad (23)$$

Interpretation. The state-pair bias asks a simple question: if the receiving node is in state u and the source node is in state v , is this kind of edge generally plausible? It is a global, non-temporal compatibility prior. In future work this may be extended into a soft or hard state-pair support mask.

8 Convex dynamic-graph gate

The gate is a safety valve on how much the dynamic graph is allowed to override the stable base graph early in training, when the dynamic scorer is still noisy. A single learned weight α controls the blend.

The dynamic-base model can be equipped with a residual gate α . In logit-gated form,

$$A_t = \text{softmax}\left(L^{\text{base}} + \alpha L_t^{\text{dyn}}\right). \quad (24)$$

In the current preferred convex form, the graph is mixed after normalisation:

$$A_t^{\text{base}} = \text{softmax}(L^{\text{base}}), \quad (25)$$

$$A_t^{\text{full}} = \text{softmax}(L^{\text{base}} + L_t^{\text{dyn}}), \quad (26)$$

$$A_t = (1 - \alpha)A_t^{\text{base}} + \alpha A_t^{\text{full}}. \quad (27)$$

The gate can be scalar or per head. It is parameterised by a raw variable $\tilde{\alpha}$ with

$$\alpha = \text{sigmoid}(\tilde{\alpha}). \quad (28)$$

The configured initial value is interpreted as the initial value of α , not as the raw sigmoid parameter.

Interpretation. The convex gate gives α a direct meaning: it is the final adjacency mixture weight. At $\alpha = 0$, the model uses the base graph. At $\alpha = 1$, it uses the full dynamic-base graph. In successful runs, α starts near 0.5, rises in phases, and tracks improvements in accuracy and graph correlation. This suggests that the gate acts as a learned dynamic-graph exposure schedule: the model opens access to dynamic structure when doing so is useful and stable.

9 Spatial message passing

Given a graph from the scorer, the spatial module is where information actually moves between nodes: each node replaces (part of) its representation with a weighted average of its neighbours' values, the weights being the inferred adjacency. This is the "use the graph" half of the architecture, kept separate from the "find the graph" half above.

The spatial module is GAT-style value aggregation with externally supplied graph weights. Given a graph $A_{b,t,h,i,j}$ and value vectors $v_{b,t,j}$, the message for target node i is

$$m_{b,t,h,i} = \sum_{j=1}^N A_{b,t,h,i,j} v_{b,t,j}. \quad (29)$$

The per-head messages are combined and passed through output and feed-forward layers with residual connections and normalisation.

Clarification. The graph scorer computes the attention-derived adjacency. The spatial layer does not compute a second independent attention score. It uses the supplied adjacency to aggregate values. Thus the architecture factorises graph inference and message passing:

$$A_t = \text{GraphScorer}(H_t), \quad H'_t = \text{SpatialMessagePassing}(H_t, A_t). \quad (30)$$

This is related to GAT-style attention, but the adjacency is a first-class object that can be logged, regularised, and compared to the true graph.

10 Interlaced spatio-temporal architecture

The original single-block architecture had the form

$$\text{Embedding} \rightarrow \text{Temporal} \rightarrow \text{Graph} \rightarrow \text{Spatial} \rightarrow \text{Head}. \quad (31)$$

This asks the graph scorer to infer the active graph before the hidden states have incorporated much cross-node evidence. The interlaced architecture repeats temporal, graph, and spatial processing:

$$H^{(0)} = \text{Embed}(s), \quad (32)$$

$$\tilde{H}^{(\ell)} = \text{Temporal}^{(\ell)}(H^{(\ell)}), \quad (33)$$

$$A^{(\ell)} = \text{GraphScorer}^{(\ell)}(\tilde{H}^{(\ell)}), \quad (34)$$

$$H^{(\ell+1)} = \text{Spatial}^{(\ell)}(\tilde{H}^{(\ell)}, A^{(\ell)}). \quad (35)$$

The current notebook uses an interlaced stack with a static first spatial module and dynamic-base later modules:

$$\text{static context graph} \rightarrow \text{dynamic graph recovery layers}. \quad (36)$$

Interpretation. The first layer often has near-maximal graph entropy. This is not necessarily a failure. It appears to act as a broad context aggregator. Later layers, especially the selected final dynamic-base layer, recover the sparse dynamic graph. This division of labour is a useful feature of the interlaced architecture.

11 Prediction objective

The model predicts the next state for every node and every timestep except the last. If $z_{b,t,i} \in \mathbb{R}^K$ denotes the predicted logits for $s_{b,t+1,i}$, the main loss is multiclass cross-entropy:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{BN(T-1)} \sum_{b=1}^B \sum_{i=1}^N \sum_{t=1}^{T-1} \log \text{softmax}(z_{b,t,i})_{s_{b,t+1,i}}. \quad (37)$$

Interpretation. Because $K = 32$, the uniform loss is $\log(32) \simeq 3.466$. A loss around 2.5 is therefore not high; it indicates substantial structure has been learned. The oracle NLL is around 2.056, so progress should be measured relative to that oracle, not relative to zero.

12 Graph regularisation

The learned graph is trained indirectly through prediction loss. Without regularisation, the model has no explicit reason to make the graph sparse, smooth, or interpretable. The current implementation therefore includes optional graph-shape regularisation applied to a selected graph layer, usually the final non-null graph:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_H (H(A) - H_\star)^2 + \lambda_S \frac{1}{T-2} \sum_{t=2}^{T-1} \|A_t - A_{t-1}\|_F^2 + \lambda_E H(A). \quad (38)$$

Here $H(A)$ is the mean row entropy of the selected graph, $H_\star$ is a target entropy, and the smoothness term discourages temporal graph flicker. Direct entropy minimisation is available but is usually left off initially. A warmup schedule scales the regularisation during early epochs.

Interpretation. The target entropy is a soft inductive bias, not graph supervision. The true graph entropy in the current generator is about 1.78. A target around 2.0 or 2.2 encourages sharper graphs without forcing the model to be as sharp as the true graph. This matters because the learned model may need to hedge over uncertain regimes.

Example observation. In one example run without regularisation, the final graph entropy ended around 2.6 with accuracy about 0.30 and graph correlation about 0.75. In a target-entropy regularised run, entropy fell to about 2.14 while accuracy and graph correlation remained very similar. The result suggests that entropy regularisation can produce a substantially cleaner graph without materially damaging the predictive solution. These figures are illustrative; later runs have improved on them, and the target remains correlation above 0.8 with entropy below 2.

13 Evaluation metrics

The model is evaluated along both predictive and structural dimensions.

13.1 Predictive metrics

The main predictive metrics are next-state accuracy and cross-entropy/NLL. Accuracy is read against the achievable ceiling, not against one. In the current benchmark the dynamic oracle accuracy is about 0.393; an example learned run reaching about 0.30 is therefore already meaningful, and the aim is to push well above that toward the ceiling.

13.2 Graph recovery metrics

For learned graph $\hat{A}_t$ and true graph $A^{(r_t)}$, graph recovery is measured using mean-squared error, correlation, and edge AUROC. Graph correlation is a particularly useful indicator of whether the inferred adjacency tracks the true dynamic structure. However, it must be interpreted alongside graph entropy.

13.3 Layer-wise graph diagnostics

In interlaced mode, each layer has its own graph. The implementation logs both selected-graph metrics and layer-wise metrics. The selected graph is typically the final dynamic graph, controlled by the graph-evaluation layer setting. Early layers should not necessarily be judged by the same criteria as the final graph.

Interpretation. A high-entropy first graph and a lower-entropy high-correlation final graph is a healthy pattern. It suggests that the early graph builds context while the later graph performs dynamic recovery.

14 Configuration

The runnable configuration lives in `ModelConfig (utilities.py)` and in the notebook, which should be treated as the source of truth for runnable values. A specific recommended configuration is deliberately *not* reproduced here: finding a setting that captures the dynamic headroom while keeping the graph faithful and selective is the exercise. The dials available to the reader — graph scorer type and activation, what the spatial layer mixes, interlacing depth, the dynamic-base gate, and the regularisation terms — are enumerated in the ablation protocol and run checklist below.

15 Ablation protocol

A useful ablation programme should change one major factor at a time. The following ladder is recommended.

15.1 Graph-type ladder

Compare:

1. no spatial graph;
2. static graph;

3. dynamic graph;
4. dynamic-base graph;
5. oracle graph.

The oracle graph is diagnostic only. It is not a trainable model. Its role is to quantify the best possible performance when the active graph is known.

15.2 Interlacing ablation

Compare the single-block architecture to interlaced variants:

Temporal $\rightarrow$ DynamicBase $\rightarrow$ Head,
 Temporal $\rightarrow$ Static $\rightarrow$ Temporal $\rightarrow$ DynamicBase $\rightarrow$ Head,
 Temporal $\rightarrow$ DynamicBase $\rightarrow$ Temporal $\rightarrow$ DynamicBase $\rightarrow$ Head.

The key question is whether cross-node context before graph scoring improves dynamic recovery.

15.3 Gate ablation

Compare no gate, fixed gate, scalar learned gate, and per-head learned gate:

$\alpha = 1$ (no gate),
 $\alpha = 0.5$ (fixed),
 $\alpha = \text{sigmoid}(\tilde{\alpha})$ (scalar learned),
 $\alpha_h = \text{sigmoid}(\tilde{\alpha}_h)$ (per-head learned).

If learned α improves stability or final performance, it is evidence that the model benefits from controlling when dynamic structure takes over.

15.4 Entropy regularisation ablation

Compare no regularisation to target-entropy regularisation. The key metrics are final entropy, effective neighbours, accuracy, NLL, and graph correlation. The desired result is not necessarily higher accuracy. A lower-entropy graph with unchanged accuracy and correlation is still useful because it is more selective and interpretable.

15.5 Capacity ablation

After a strong configuration is identified, reduce capacity one element at a time:

1. reduce graph hidden dimension;
2. reduce model dimension;
3. reduce temporal layers;
4. reduce edge heads;
5. reduce number of interlaced blocks.

Do not compress before reproducing the known strong run. Otherwise it is difficult to know whether a failure is architectural or simply under-capacity.

16 Practical guidance for running experiments

Use the same data when comparing models. The synthetic dataset should be generated once for a ladder so that differences are due to model configuration rather than data variation.

Read accuracy against the oracle ceiling. An example model accuracy around 0.30 already beats the static oracle (≈ 0.230) and captures part of the headroom toward the dynamic oracle (≈ 0.393); a strong configuration should push closer to the dynamic ceiling.

Inspect the alpha trajectory. If the convex gate opens gradually and the opening tracks improvements in correlation and accuracy, the gate is behaving as a stability mechanism. If alpha immediately saturates, compare against no-gate. If alpha remains near zero, the dynamic graph is not being used.

Inspect graph entropy and effective neighbours. The true graph has about six effective neighbours. A learned graph with entropy near 3.3 is too diffuse. A graph around 2.1 to 2.6 may be a useful compromise depending on prediction and recovery metrics.

Do not regularise every layer by default. Early layers may serve as broad context aggregators. Regularise and evaluate the final selected dynamic graph first.

Use layer-wise diagnostics. A healthy interlaced model can show high entropy in layer 0 and much lower entropy with high correlation in the final layer. This is a feature, not necessarily a bug.

17 Future extension: state-pair support masks

The current state-pair bias is an additive scalar lookup table. A natural next extension is a state-pair support prior. The simplest version introduces a learned mask

$$M_{h,u,v} = \text{sigmoid}(B_{h,u,v}), \quad (39)$$

and adds it to the edge logits as a log prior:

$$L_{b,t,h,i,j} \leftarrow L_{b,t,h,i,j} + \beta \log (M_{h,s_{b,t,i},s_{b,t,j}} + \epsilon). \quad (40)$$

This factorises graph learning into a global state-pair compatibility prior and a context-dependent dynamic graph. Later versions may use straight-through, Gumbel, or top- k masks to remove irrelevant connections while preserving gradients.

Interpretation. This extension asks whether some source-target state pairs are globally implausible regardless of the current regime. If so, the dynamic graph scorer should not have to rediscover this support constraint from temporal context alone.

18 Summary

BaseDyGraph is useful because it combines a controlled data generator, oracle headroom diagnostics, interpretable dynamic graph recovery metrics, and production-style architectural components. The current results support several working hypotheses. Interlaced spatio-temporal processing allows later graph scorers to use cross-node context. A convex base-to-dynamic gate stabilises the transition from average static structure to dynamic graph use. Graph entropy regularisation can produce cleaner graphs without materially degrading accuracy or correlation. The model should therefore be studied not by a single accuracy number, but by the joint behaviour of predictive loss, dynamic headroom captured, graph correlation, graph entropy, layer-wise diagnostics, and alpha trajectories.

References

- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. Graph attention networks. *International Conference on Learning Representations*, 2018.
- Wu, Z., Pan, S., Long, G., Jiang, J., and Zhang, C. Graph WaveNet for deep spatial-temporal graph modeling. *International Joint Conference on Artificial Intelligence*, 2019.
- Martins, A. F. T. and Astudillo, R. F. From softmax to sparsemax: A sparse model of attention and multi-label classification. *International Conference on Machine Learning*, 2016.
- Peters, B., Niculae, V., and Martins, A. F. T. Sparse sequence-to-sequence models. *Association for Computational Linguistics*, 2019.

A Glossary for new researchers

Dynamic graph. A graph whose edge weights change over time or context. In this benchmark, the true dynamic graph changes when the latent regime changes.

Static graph. A graph that is fixed across all timesteps. It can still be a strong predictor because it captures average dependency structure.

Dynamic headroom. The performance gap between a dynamic oracle and a static oracle. It measures what can only be gained by recovering time-varying structure.

Graph entropy. The entropy of each row of the learned adjacency. It measures how many neighbours receive substantial mass. Lower entropy means a sharper graph.

Effective neighbours. The exponential of graph entropy. It converts entropy into an approximate number of active neighbours.

Convex alpha gate. A learned mixture weight between the base graph and the full dynamic graph. It provides a stable route from static structure to dynamic structure.

Interlacing. Alternating temporal and spatial processing blocks. Interlacing allows later graph scorers to use hidden states that already contain cross-node information.

B Checklist for an ablation run

Before starting a run, record the following:

1. data seed and synthetic generator configuration;
2. model dimension, graph hidden dimension, number of temporal layers, number of ST blocks;
3. graph scorer type and graph activation;
4. whether dynamic-base convex gating is enabled;
5. initial alpha and whether alpha is learnable;
6. graph regularisation target, coefficient, layer, and warmup;
7. W&B group name and run name.

At the end of a run, compare:

1. validation accuracy and prediction loss;
2. graph correlation and graph AUROC;
3. graph entropy and effective neighbours;
4. alpha trajectory;
5. layer-wise graph entropy and graph correlation;
6. fraction of dynamic headroom captured.